



KUBERNETES MONITORING WITH PROMETHEUS & GRAFANA

Day 16
K8s Series

★ Observe. Understand. Optimize. ★

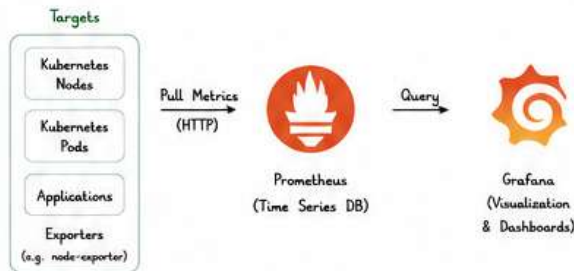
1 What is Monitoring?

- Monitoring helps us track the health, performance and availability of Kubernetes clusters and applications.
- It provides insights through metrics, logs and alerts.
- Prometheus collects metrics.
- Grafana visualizes metrics.

In Simple Words

Prometheus collects the data, Grafana makes sense of the data.

2 Monitoring Architecture



How it works:

- Prometheus scrapes metrics from configured targets.
- Stores metrics as time series data.
- Grafana queries Prometheus and shows dashboards.
- Alerts can be triggered based on rules.

3 Prometheus Components

Component	Purpose
Prometheus Server	Collects, stores and queries metrics
Exporters	Expose metrics from systems/apps (e.g. node-exporter)
Service Discovery	Finds targets dynamically (e.g. Kubernetes SD)
Alertmanager	Handles alerts (grouping, routing, notifications)
Rules	Define alerting and recording rules

Key Point

★ Prometheus follows a pull model.
It pulls metrics from targets at regular intervals.

4 Key Metrics to Monitor

- Node: CPU, Memory, Disk, Network
- Pod: CPU Usage, Memory Usage, Restarts
- Kubernetes: API Server, Scheduler, Controller Manager
- Cluster: etcd, kubelet, kube-proxy
- Applications: Request rate, Error rate, Latency

Remember!



Monitor RED (Rate, Errors, Duration) for your applications.

5 Prometheus - Installation (Helm)

```

# Add repo
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update

# Install kube-prometheus-stack
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace

# Check status
kubectl get pods -n monitoring
  
```

kube-prometheus-stack includes Prometheus, Alertmanager, Grafana, Node Exporter, Kube State Metrics and more.



6 Accessing Prometheus & Grafana

- Port-forward services (for local access)

```

kubectl port-forward svc/monitoring-kube-prometheus-prometheus \
  -n monitoring 9090:9090

kubectl port-forward svc/monitoring-grafana \
  -n monitoring 3000:80
  
```

- URLs

Prometheus: <http://localhost:9090>
Grafana : <http://localhost:3000>

Default Grafana Login:
username: admin
password: prom-operator



7 Grafana - Dashboards

- Grafana provides beautiful dashboards.
- Pre-built dashboards available for:
 - Kubernetes / Nodes
 - Pods
 - Prometheus
 - Applications



Popular Dashboard IDs

- Kubernetes Cluster: 315
- Kubernetes Pods: 6417
- Node Exporter Full: 1860
- Prometheus Stats: 2

8 PromQL Basics (Prometheus Query Language)

Query	Description
up	Targets that are UP (1) or DOWN (0)
rate(http_requests_total[5m])	Request rate per second
sum(rate(container_cpu_usage_seconds_total[5m])) by (pod)	CPU usage by pod
sum(container_memory_usage_bytes) by (pod)	Memory usage by pod
increase(http_requests_total[1h])	Total increase in last 1 hour
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))	95th percentile latency



Tip: Use PromQL in Prometheus UI → Graph tab to explore metrics.

9 Alerting with Prometheus

- Define alert rules based on conditions.
- Alerts are sent to Alertmanager.
- Alertmanager routes notifications to channels (Email, Slack, PagerDuty, etc.)

Example Alert Rule (YAML)

```

groups:
- name: high-cpu
  rules:
  - alert: HighCPUUsage
    expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]) * 100) > 80
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "High CPU usage on {{ $labels.instance }}"
      description: "CPU usage is above 80% for more than 5 minutes."
  
```



10 Monitoring Components in kube-prometheus-stack

Component	Purpose
Prometheus	Scrapes and stores metrics
Grafana	Visualization and dashboards
Alertmanager	Alert handling and notifications
Node Exporter	Node level metrics
Kube State Metrics	Kubernetes object metrics
Prometheus Operator	Manages Prometheus instances
kubelet / cAdvisor	Container and node metrics

11 Important kubectl Commands

```

kubectl get pods -n monitoring
kubectl get svc -n monitoring
kubectl get prometheus -A
kubectl get alertmanagers -A
kubectl describe pod <pod> -n monitoring
kubectl logs <pod> -n monitoring
kubectl top nodes
kubectl top pods

# List monitoring pods
# List monitoring services
# List Prometheus instances
# List Alertmanagers
# Describe a pod
# Logs of a pod
# Node resource usage
# Pod resource usage
  
```

12 Troubleshooting Tips

- Targets DOWN in Prometheus?**
 - Check service discovery & network connectivity.
- No metrics showing?**
 - Check if exporters are running.
- Grafana no data?**
 - Verify data source and PromQL query.
- High CPU/Memory on Prometheus?**
 - Adjust scrape interval & retention.



13 Best Practices

- Monitor everything important.
- Use meaningful dashboards.
- Set alerts for critical issues.
- Keep alerts actionable (avoid noise).
- Use recording rules for complex queries.
- Monitor the monitoring (Prometheus itself).



14 Real World Use Cases



Detect node failures



Track application performance



Find bottlenecks (CPU/Memory)



Capacity planning



SLA & Error tracking

15 Key Takeaway

Prometheus collects the metrics.
Grafana turns those metrics into insights.
Together, they help you build reliable, performant and observable Kubernetes applications.



Good monitoring doesn't just show you problems, it helps you prevent them!

